VINAY.T

GIT Interview Questions
DAY 2

Intermediate GIT Interview Questions

11.Why is Git considered an easy and efficient tool for developers to work with?

Git has revolutionized the way developers manage and collaborate on code. By offering a robust set of features tailored to modern development needs, Git enables faster, more efficient, and more reliable workflows. Below are some of the key features that have made Git a fundamental tool for developers:

1. Advanced Branching Capabilities

One of Git's most powerful features is its advanced branching system, which allows developers to create, manage, and switch between multiple branches with ease.

- Developers can work simultaneously on different features, bug fixes, or experiments without affecting the main codebase.
- Each branch serves as an isolated environment, encouraging a cleaner and more organized workflow.
- Merging branches is streamlined with Git's efficient merge options and conflict-resolution mechanisms.
- Git also supports visual tools and diagrams to help track and manage complex branch workflows, aiding in better team collaboration and transparency.

2. Distributed Development Architecture

Git is a distributed version control system, which offers several advantages over centralized systems:

- Every developer has a full copy of the repository, including its entire history, on their local machine. This is known as the local repository, while the copy on the server is referred to as the remote repository.
- This architecture increases fault tolerance—if the main server goes down or data is lost, the codebase can be restored from any team member's local copy.
- It supports offline work, allowing developers to commit and review changes without needing constant access to the central server.
- The distributed nature of Git promotes better scalability and flexibility, especially for large teams and open-source projects.

3. Pull Requests for Team Collaboration

Pull requests (PRs) are a central feature for teams using Git, especially when working on platforms like GitHub or GitLab.

- PRs provide a structured way for developers to propose changes to the codebase and seek feedback or approval from their peers before merging.
- They serve as a collaboration hub, where discussions, reviews, and automated checks (such as CI/CD pipelines) take place.
- Pull requests help maintain code quality and consistency, as every change is tracked, reviewed, and documented.
- This feature enhances communication within teams and streamlines the code integration process.

4. Efficient Release Cycle Management

Git plays a vital role in accelerating and managing release cycles in modern software development.

- Features like tagging and branching strategies (e.g., Git Flow or trunk-based development) help manage different stages of a release, such as development, testing, staging, and production.
- Teams can roll out updates more frequently and reliably thanks to Git's structured workflows.
- With a clear version history and robust change-tracking, Git ensures that releases are stable, traceable, and easier to maintain or rollback if needed.

In conclusion, Git's feature-rich ecosystem empowers developers to work more collaboratively, maintain higher code quality, and deliver software faster. Whether you're working solo or part of a large team, mastering Git is essential for modern development success.

12.Can you explain the steps involved in creating a new Git repository?

To create a Git repository from scratch, follow these steps:

a.Ensure Git is Installed

First, make sure that Git is installed on your system. You can verify this by running the following command in your terminal or command prompt:
git --version

If Git is installed, this command will display the currently installed version. If not, you'll need to download and install Git from the official website: https://git-scm.com

b.Create a Project Directory

Next, create a folder that will hold your project files. This folder will also become your Git repository.
mkdir my-project
cd my-project

Replace my-project with your desired folder name.

c.Initialize the Git Repository

Inside the project directory, run the following command:
git init

This command initializes a new Git repository in your project folder. It creates a hidden directory called .git, which contains all the configuration files and metadata Git needs to track versions of your project.

d.Confirm Repository Creation

You can verify that the repository was successfully created by listing the hidden files:
ls -a

You should see a .git directory in the list. This indicates that the folder is now a Git repository, and you can start tracking changes, committing files, and using other Git features.

13.What is the purpose of *git stash* in Git

git stash is a useful command in Git that allows developers to temporarily save their uncommitted changes without having to commit them. This is particularly helpful when you're in the middle of working on something and suddenly need to switch to a different branch to address an urgent task, bug fix, or review.

When you run the *git stash* command, Git takes the current state of your working directory (i.e., any modified or staged files) and index (the staging area), and saves it on a stack-like structure managed by Git. It then reverts your working directory to a clean state, essentially, as if there were no changes , so you can safely switch branches or perform other operations.

This allows you to:

- Avoid losing uncommitted changes.
- Quickly context-switch between tasks or branches.
- Restore your changes later when you're ready to resume work.

You can view all stashed entries with:
git stash list

And when you're ready to reapply the changes, you can use:
git stash apply
or
git stash pop

The difference is that *apply* keeps the stash in the stack, while *pop* reapplies the changes and removes the entry from the stash list.

In short, *git stash* is an excellent tool for managing workflow interruptions without cluttering the commit history or risking the loss of work.

14.Which command is used to delete a Git branch?

Deleting branches in Git is a common housekeeping task, typically done once a feature, fix, or enhancement has been successfully merged into a main branch (such as *main* or *develop*). This helps keep the repository clean and organized, reducing clutter and the chance of confusion.

1. Deleting a Local Branch

To delete a branch from your local repository, you can use the following command:
git branch -d <local_branch_name>

- The -d flag stands for delete and is a safe option that only deletes the branch if it has already been merged into the current branch or another specified branch.
- If you want to forcefully delete a branch regardless of its merge status (not recommended unless you’re sure), you can use:

git branch -D <local_branch_name>

2. Deleting a Remote Branch

To delete a branch from the remote repository, use the following command:
git push origin --delete <remote_branch_name>

- This tells Git to instruct the remote (usually named *origin*) to delete the specified branch.
- Once executed, the branch will no longer be available on the remote server, though it may still exist in other local clones until they sync with the remote.

When and Why to Delete Branches

- After Merge: A common scenario is deleting a feature or bug-fix branch after it has been successfully merged into a long-lived branch like *develop* or *main*.
- To Reduce Clutter: Regularly deleting unused or obsolete branches helps maintain a cleaner, more manageable Git history.
- Prevent Mistakes: Removing outdated branches reduces the risk of developers accidentally working on stale code.

In summary, Git provides simple yet powerful commands to delete both local and remote branches, ensuring your version control remains efficient and well-organized.

15.What is the difference between git remote and git clone commands?

Although both git remote and git clone involve working with remote repositories, they serve very different purposes within a Git workflow.

1. git clone – Cloning a Repository

The git clone command is used to create a copy of an existing remote repository. When you run this command, Git:
- Downloads the entire repository history, including all branches, tags, and commits.
- Creates a new directory on your local machine containing all the project files.
- Automatically sets up a remote named origin that points to the source repository URL.

Example:
git clone https://github.com/user/project.git

This creates a local copy of the project repository and links it to the specified remote.

2. git remote – Managing Remote Connections

The git remote command is used to manage remote repository connections for an existing local Git repository. It allows you to add, remove, rename, or list remote URLs.

Unlike git clone, this command does not create a new repository—instead, it modifies the configuration of a Git project you already have locally.
- When you use git remote add, you're assigning a name (usually origin) to a specific repository URL.
- This makes it easier to interact with remotes using shorthand names instead of full URLs.

Example:
git remote add origin https://github.com/user/project.git

This command adds a remote named origin that points to the given URL, allowing you to push and pull code from that location.

Command	Purpose	Creates New Repo?	Adds Remote?
git clone	Copies a remote repository to your local system	✅ Yes	✅ Yes
git remote	Manages remote URLs in an existing local repository	❌ No	✅ Yes

In essence:

- Use *git clone* when you want to start working on a new project by copying an existing repository.
- Use *git remote* when you want to link an existing local project to a remote repository or manage remote connections.

16.Can you explain what the git stash apply command does and when you would use it?

The git stash apply command is used to retrieve and reapply previously stashed changes to your working directory. It is part of Git's stashing feature, which allows developers to temporarily save uncommitted changes without committing them, typically when they need to switch branches or handle an urgent task without losing their current progress.

How It Works
When you run the *git stash* command, Git:

- Saves the current state of your working directory (modified but not committed files).
- Stores this snapshot in a stash stack, effectively clearing your workspace.
- Allows you to switch branches or perform other operations without the risk of losing changes.

Once you're ready to return to your previous work, you can use:
git stash apply

This command takes the most recent stash entry from the stack and reapplies the saved changes to your working directory.

Key Points:

- *git stash* apply does not remove the stash entry from the stack after applying it. If you want to apply and remove the stash, use *git stash pop* instead.
- You can also specify a particular stash if you have multiple stashed changes:
  git stash apply stash@{1}
- This command helps developers seamlessly resume their interrupted work without having to manually back up or redo their changes.

Use Case Example:
Imagine you're working on a new feature, but suddenly need to switch to the main branch to fix a critical bug. You haven’t committed your changes yet, and you don’t want to lose them.

Here's how you’d use stash:

```
git stash             # Save current changes
git checkout main # Switch to fix the bug
# Fix and commit the bug...
git checkout feature-branch
git stash apply      # Restore the stashed changes and resume work
```

Conclusion
The *git stash apply* command is a powerful tool that allows developers to quickly and safely reapply previously saved work. It supports an efficient and interruption-friendly workflow, especially when managing multiple tasks across branches.

17.What is the difference between git pull and git fetch?

Although both git pull and git fetch are used to retrieve updates from a remote repository, they serve different purposes and behave differently in terms of how they apply those updates to your local working branch.

1. git pull – Fetch and Merge in One Step
The git pull command is used to automatically download and integrate changes from a remote repository into your current working branch.

- It combines two actions:
  a. Fetches the latest commits and changes from the specified branch of the remote repository.
  b. Merges those changes directly into your current local branch.

Example:
git pull origin main

This command will fetch updates from the main branch on the remote named origin and immediately merge them into your currently checked-out branch.

2. git fetch – Fetch Only (No Merge)
The git fetch command is used to download new data from the remote repository, such as branches, commits, and tags, but does not apply these changes to your working branch automatically.

- Instead, the fetched content is stored in a separate area within your local repository.
- To apply the updates to your current branch, you must explicitly run a git merge or git rebase afterward.

Example:
git fetch origin
git merge origin/main

This approach gives you more control over when and how changes are merged into your local branch, which is helpful for reviewing updates before applying them.

Key Differences:

Feature	gitpull	git fetch
Downloads remote data	✅ Yes	✅ Yes
Automatically merges	✅ Yes	❌ No (requires manual merge or rebase)
Control over changes	❌ Less control	✅ Full control
Common usage	Quick updates to local branch	Review changes before integrating them

Summary

- Use git pull when you want to quickly update your current branch with the latest changes from the remote repository.
- Use git fetch when you want to see what has changed in the remote before applying those changes, giving you a safer and more controlled workflow.

In essence:
git pull = git fetch + git merge

This distinction is important, especially when collaborating in teams or managing large codebases where you want to minimize unintended merges or conflicts.

18.What is the difference between a branch and a pull request in Git

Both branches and pull requests are fundamental to collaborative development using Git, especially in platforms like GitHub, GitLab, or Bitbucket. However, they serve very different purposes in the software development workflow.

1. Branch – A Separate Line of Development
A branch in Git is essentially an independent version of the codebase. It allows developers to isolate their work without affecting the main codebase (often the main or master branch).

- Branches are typically used for developing new features, fixing bugs, or experimenting with code.
- Multiple developers can work on different branches in parallel.
- Branches help manage the development workflow by keeping changes separate until they are ready to be reviewed and merged.
-

Example:
git checkout -b feature/login-page

This creates a new branch called feature/login-page based on the current branch.

2. Pull Request – A Request to Merge Changes
A pull request (often abbreviated as PR) is a formal request to merge one branch into another, usually after the work is complete and tested.

- It is created after changes have been pushed to a branch.
- Pull requests are typically reviewed by team members before being merged, ensuring code quality and alignment with project standards.
- They often include discussion threads, automated test results, and change comparisons.

In essence, a pull request is not a Git command, but a feature provided by Git hosting platforms like GitHub, GitLab, and Bitbucket.

Typical flow:

1. Developer creates a branch and makes changes.
2. Pushes the branch to a remote repository.
3. Opens a pull request to merge the feature branch into main or another target branch.

Summary of Key Differences

Aspect	Branch	Pull Request
Definition	A separate line of development in a Git repository.	A request to merge changes from one branch into another.
Purpose	To isolate code changes for a specific task.	To review and merge changes into a main or shared branch.
Used By	Developers during development.	Developers and reviewers during code collaboration.
Built-in Git Feature?	Yes	No (it's a platform feature, e.g., GitHub, GitLab)
Requires Review?	No	Typically yes (review, testing, and approval before merge)

Conclusion

- A branch is where the actual development happens—it's your workspace.
- A pull request is how you propose to share your work with the team and integrate it into the main project after review.

Together, they support efficient, organized, and collaborative development in modern version control workflows.

19.Why do we not call git “pull request” as “push request”? **VINAY.T**

The term “pull request” may initially seem counterintuitive because developers are pushing their changes to a repository, so one might expect it to be called a “push request.” However, the naming is based on how the changes are intended to be integrated by the target repository, not the action of the contributor.

Understanding the Terminology
- A pull request is essentially a request to the maintainers of a target branch (typically in a shared or main repository) to pull your changes from your branch into theirs.
- It signals that you’ve completed work on your branch and are now asking the maintainers to review and pull your changes into their codebase.

Why Not “Push Request”?
- A push in Git is a direct action that sends your commits to a remote repository, and it assumes you have permission to do so.
- If contributors could simply push to a main branch, it would bypass the review and collaboration process.
- In collaborative environments (like GitHub, GitLab, Bitbucket), contributors typically do not have direct write access to the main repository. Instead, they fork or branch the project, make their changes, and then request the maintainers to pull the changes via a pull request.

How a Pull Request Works:
1. A developer makes changes on their own branch or fork.
2. They push their changes to a remote repository under their control.
3. They open a pull request to the target repository.
4. The maintainers of the target repository review the request and pull the changes if approved.

Summary
- A pull request is named from the perspective of the target repository: you are requesting them to pull your changes.
- A “push request” would imply you’re directly sending changes into someone else’s codebase, which bypasses the standard collaborative and review process.

This naming emphasizes a controlled and review-based workflow, which is essential in team-based or open-source development environments.

20.What is the difference between Git and GitHub?

Although Git and GitHub are closely related and often used together, they serve very different purposes in the software development ecosystem. Here's a comprehensive breakdown of their roles and distinctions:

1. Git – Distributed Version Control System
- Definition: Git is an open-source, distributed version control system (DVCS) that helps developers track and manage changes in their source code over time.
- Usage: Installed locally on developers' machines, Git allows multiple developers to work independently on different branches and later merge their work into a single source of truth.
- Functionality:
  - Tracks commit history
  - Supports branching and merging
  - Enables offline development
  - Resolves code conflicts
- Maintained By: The Linux Foundation
- Competitors: SVN (Subversion), Mercurial, Perforce

2. GitHub – Cloud-Based Hosting Platform for Git Repositories

Definition: GitHub is a cloud-based platform built around Git that allows developers and teams to host, share, and collaborate on Git repositories.

Functionality:

- Provides a web-based graphical interface for Git
- Supports collaboration through pull requests, code reviews, and issue tracking
- Offers services like forking, repository permissions, CI/CD integrations, and more
- Allows access to remote repositories from anywhere with an internet connection

Ownership: Acquired by Microsoft in 2018

Competitors: GitLab, Bitbucket (Atlassian), Azure DevOps, SourceForge

Summary

- Git is the underlying tool used to manage version control locally.
- GitHub is a collaboration and code-hosting platform that builds on Git's capabilities, offering remote repository access, team collaboration features, and automation tools.

In simple terms:
Git is the engine, GitHub is the garage.
Git manages your code changes, while GitHub provides a platform to store, share, and collaborate on that code.